★ FINAL YEAR B.TECH CAPSTONE PROJECT SPECIFICATION & ARCHITECTURE HANDBOOK ★

SmartVision: AI Multi-Face Biometric Attendance Portal & Emergency Proxy Reallocation Desk

A Deep Learning Computer Vision Platform with ResNet-34 128-D Feature Embeddings, WebRTC Live Video Scanning, Automated Emergency Substitute Reallocation, Geofenced Faculty Punching & Dynamic Retentive Risk Analytics

---

Project Domain:	Computer Vision, Deep Metric Learning, WebRTC Streaming, Institutional ERP
Core Vision Algorithm:	Dlib ResNet-34 128-D Embeddings, HOG Frontal Locator, 68-Point Landmark Warper, OpenCV
Backend Web Framework:	Python 3.14/3.11, Flask 3.1 (Modular Blueprints: main, teacher, student, auth, teacher_attendance)
Database & Persistence:	SQLAlchemy 2.0 ORM, 31 Relational Models, SQLite (Dev) / PostgreSQL (Production)
Frontend & Streaming:	WebRTC navigator.mediaDevices API, HTML5 Canvas 2D Overlay, Bootstrap 5.3, Vanilla CSS3
Security & Identity:	PBKDF2:SHA256 (600,000 rounds), Google OAuth 2.0 SSO (Authlib), Role-Based Scoping
Author / Lead Developer:	**Shivam Vishwakarma** (CSE Dept., PIET, Parul University)
Project Guide:	Prof. Harsh Pateliya (Assistant Professor, Department of Computer Science & Engineering)
Target Evaluation:	Final Year External University Examination & Technical Viva Voce Defense

Handbook Purpose & Architecture Overview:
This master handbook provides exhaustive structural, architectural, algorithmic, and operational documentation for the SmartVision Attendance Portal. It is structured specifically so that candidates and evaluators can dissect every file, class, function, route, database model, mathematical formula, deployment pipeline, and Viva question with complete technical depth.

1. System Architecture & Complete Data Flow Mindmap

SmartVision operates on a decoupled 4-Tier Architecture connecting the browser client to the AI vision pipeline, timetable scheduling engine, and relational persistence layer:

```
==================================================================================================== SMARTVISION
ARCHITECTURE & FLOW MINDMAP
==================================================================================================== [CLIENT BROWSER
LAYER] ■■■ WebRTC Stream (navigator.mediaDevices.getUserMedia) ■■■■■■■■■■■■■■■■ (Base64 JPEG Frames) ■■■ HTML5 Canvas
Hardware-Accelerated Bounding Box Overlay ■■■■■■■■■■■■■ (JSON Face Coordinates) ■■■ Multi-Role Interfaces (Admin,
Teacher, Class Teacher, Student) ■ ■■■ Google OAuth 2.0 SSO Sign-In / SMTP OTP Multi-Factor Verification ■ ▼
[APPLICATION & ROUTING LAYER (Flask Blueprints)] <■■■■■■■■■■■■■■■■■■■■■■■■■■ ■■■ auth/routes.py (21 Functions: OAuth,
Login, Register, Camera Photo Registration) ■■■ main/routes.py (73 Functions: Admin Dashboard, Proxy Desk, Approvals,
live_detect API) ■■■ teacher/routes.py (32 Functions: Live WebRTC Attendance, Group Photo Scanner, Proxy) ■■■
student/routes.py (8 Functions: Dynamic Subject Analytics, Discrepancies, Notices) ■■■ teacher_attendance/routes.py (13
Functions: Haversine Geofencing, Biometric Punch) ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ▼ ▼
[AI VISION & DEEP LEARNING ENGINE] [CORE BUSINESS SERVICES & ENGINES] ■■■ 1. Frame Ingestion & Bilinear Resize ■■■
schedule_service.py (Timetable Expander) ■■■ 2. OpenCV Haar Cascade / HOG Face Locator ■■■ Strict Attendance Formula
Calculator ■■■ 3. 68-Point Facial Landmark Shape Predictor ■■■ Emergency Proxy Workload Matrix ■■■ 4. ResNet-34 128-D
Deep Metric Embeddings ■■■ email_utils.py (SMTP HTML Dispatcher) ■■■ 5. NumPy Vectorized Euclidean Distance ■■■
ai_copilot_service.py (Admin Copilot) ■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ▼ [DATA &
PERSISTENCE LAYER (SQLAlchemy 2.0 ORM)] ■■■ 31 Relational DB Models (models.py) with Cascading Deletes & Foreign Key
Integrity ■■■ Binary BLOB Embeddings: float64[128] arrays serialized into raw 1,024-byte chunks ■■■ Storage Engines:
SQLite (instance/smartvision.db) / PostgreSQL (Production Cloud)
====================================================================================================
```

Detailed Data Journey (Class Attendance Session):

1. **Session Initialization:** Teacher opens a scheduled lecture period (e.g. Period 2: 7CSE4 Computer Networks). The system retrieves all enrolled students and pre-fetches their 128-D facial vectors into RAM.
2. **Live Video / Snapshot Ingestion:** WebRTC streams video frames to the client canvas. The client dispatches 350ms frame payloads to /api/live_detect.
3. **Face Extraction & Vector Alignment:** The server detects face coordinates, warps the face using 68 landmarks, and computes the 128-D embedding vector.
4. **Parallel NumPy Vector Matching:** The vector is subtracted from all enrolled student vectors simultaneously. The minimum Euclidean distance yields the matched roll number.
5. **Persistence & Audit Log:** Attendance records are inserted with status PRESENT / ABSENT, confidence scores, and timestamps.

2. Technology Stack & Architectural Decision Matrix

Every library, protocol, and database engine in the SmartVision portal was selected based on strict empirical tradeoffs regarding inference latency, memory efficiency, and developer productivity:

Component / Layer	Technology Chosen	Alternative Evaluated	Why Chosen (Technical Rationale & Tradeoff)
Backend Web Framework	**Flask 3.1** (WSGI Microframework)	Django, Node.js Express, FastAPI	• **Low CPU Overhead:** Flask provides minimal abstraction layers, crucial when running multi-face matrix operations on CPU instances. • **Modular Blueprints:** Allows clean separation into 5 distinct routing domains without monolithic boilerplate. • **Direct C-Binding Integration:** Seamless interoperability with Dlib and OpenCV native extensions.
Deep Metric Learning Model	**ResNet-34** (128-D Metric Space)	LBPH, Haar Cascades only, FaceNet 512-D	• **99.38% LFW Accuracy:** Triplet-loss trained ResNet-34 captures invariant facial contours (eyes, nose, jawline) under varying illumination and head angles. • **Compact 128-D Size:** Produces 128 floats (1,024 bytes) compared to 512-D vectors, cutting memory usage by 75% while maintaining state-of-the-art accuracy.
Face Locator & Alignment	**HOG + 68-Point Shape Predictor**	MTCNN, SSD MobileNet, OpenCV Haar only	• **Sub-50ms CPU Execution:** Histogram of Oriented Gradients (HOG) operates at high speed on dual-core CPUs. • **Affine Geometric Warping:** 68-point landmarks rotate tilted faces upright before encoding, preventing angle-induced misclassifications.
Live Video Capture	**WebRTC API** (getUserMedia)	Flash, WebSocket raw streams, File upload only	• **Zero-Plugin Native Standard:** Works across desktop and mobile browsers (iOS Safari, Android Chrome). • **Hardware Accelerated:** HTML5 Canvas 2D context draws bounding boxes at 30 FPS with zero DOM reflow overhead.
Database ORM & Storage	**SQLAlchemy 2.0** SQLite & PostgreSQL	Raw SQL, MongoDB, Firebase	• **Binary BLOB Optimization:** 128-D arrays stored as raw binary (db.LargeBinary), deserialized instantaneously via np.frombuffer. • **Relational Integrity:** Foreign keys enforce timetable, session, student, and proxy constraints.
Authentication & SSO	**PBKDF2:SHA256** + Google OAuth 2.0	Plain SHA256, JWT in LocalStorage	• **600,000 Iterations:** Resistant to brute-force GPU dictionary attacks. • **Secure HTTPOnly Cookies:** Prevents XSS token theft. • **Institutional SSO:** Single-click Google login for faculty and students.

3. Complete Codebase Anatomy: Core Root Modules

Below is the structural anatomy of all root modules, detailing internal classes, functions, line counts, and operational responsibilities:

File Name & Path	Lines & Size	Internal Classes & Key Functions	Exact Architectural Responsibility
app.py	301 lines 15.2 KB	• create_app() • setup_initial_data()	Application factory: initializes extensions (SQLAlchemy, LoginManager, OAuth), registers 5 blueprints, registers custom Jinja date/time filters, runs database auto-migrations, and handles dev server execution on 0.0.0.0:7860.
models.py	645 lines 34.1 KB	**31 Model Classes:** User, Teacher, Student, Class, Subject, Timetable, DailySchedule, AttendanceSession, AttendanceRecord, TeacherDailyAttendance, TeacherLeave, CorrectionRequest, ProxyAttendanceTransfer, ClassAnnouncement, etc.	Defines complete relational database schema. Implements password hashing methods, relationship cascading (delete-orphan), binary face vector storage, timetable status enums, and audit trails.
config.py	55 lines 2.2 KB	• class Config	Central configuration class: extracts environment variables from .env with secure defaults for database URIs, secret keys, upload folder paths, mail server parameters, and Google OAuth credentials.
extensions.py	72 lines 2.8 KB	• db (SQLAlchemy) • login_manager • oauth • get_current_date() • get_current_datetime_str()	Instantiates shared Flask extensions without circular dependencies. Provides standardized Indian Standard Time (IST / Asia/Kolkata) date/time helpers across all blueprints.
schedule_service.py	149 lines 6.3 KB	• generate_daily_schedule() • calculate_student_attendance()	1. Expands master recurring timetable entries into date-specific DailySchedule rows, resolving holidays, approved teacher leaves, and proxies. 2. Computes the strict attendance formula: (Present / Completed Sessions) * 100.
email_utils.py	65 lines 2.5 KB	• send_email()	Asynchronous SMTP client using Python's email.mime and smtplib. Formats and sends responsive HTML transactional emails for verification OTPs, password resets, and proxy alerts.
db_migrations.py	165 lines 7.1 KB	• run_migrations()	Idempotent schema migrator: inspects live SQLite/PostgreSQL tables on application startup and executes ALTER TABLE ADD COLUMN statements for newly added model fields without dropping existing data.
ai_copilot_service.py	850 lines 36.4 KB	• ask_ai_copilot() • build_role_scoped_context() • handle_defaulters_query() • handle_admin_teacher_query()	Administrative AI Copilot: processes natural language queries regarding student attendance, faculty workloads, timetables, and retention risks with role-scoped security boundaries.

4. Complete Codebase Anatomy: Flask Blueprints & Subsystems

The application is divided into 5 specialized blueprints handling authentication, administration, teacher workflows, student self-service, and faculty biometric attendance:

Blueprint Path	Lines & Size	Core Functions & Routes	Subsystem Scope & Functionality
main/routes.py (Admin Blueprint)	3,423 lines 142.7 KB	**73 Endpoints:** • compute_emergency_proxy_desk() • dashboard() • emergency_proxy_desk() • assign_slot_proxy() • cancel_class_slot() • restore_class_slot() • api_live_detect() • retention_risk_report() • manage_timetable() • admin_approvals()	Central Administrative Control Center: executes emergency workload transfers for absent faculty, resolves free teacher matrices, processes real-time WebRTC face detection frames (/api/live_detect), generates retention risk reports (5-day absence threshold), manages institutional timetables, approves student edits, and oversees college-wide master data.
teacher/routes.py (Teacher Blueprint)	1,860 lines 79.5 KB	**32 Endpoints:** • take_attendance() • confirm_attendance() • quick_add_roll() • proxy_classes() • take_proxy_attendance() • leave_applications() • teacher_reports() • student_modifications()	Teacher Lecture & Attendance Engine: manages class-scoped attendance sessions, integrates WebRTC live camera scanner and multi-image group photo uploader, provides interactive manual roll checklist, supports same-day attendance editing (before 11 PM IST), handles proxy duty classrooms, and enables one-click WhatsApp attendance report sharing.
student/routes.py (Student Blueprint)	410 lines 17.8 KB	**8 Endpoints:** • dashboard() • edit_profile() • report_discrepancy() • request_modification() • notices() • dismiss_notice()	Student Portal & Analytics: displays real-time subject-wise attendance percentages, highlights 75% exam eligibility debarment risks, provides formal discrepancy dispute submission with proof uploads, allows profile correction requests, and broadcasts class announcements and cancelled lecture notices.
auth/routes.py (Auth Blueprint)	1,072 lines 51.6 KB	**21 Endpoints:** • login() • admin_login() • register() • google_login() • google_callback() • verify_email_link() • reset_password() • logout()	Security & Authentication Subsystem: multi-role authentication with session isolation, Google OAuth 2.0 Single Sign-On, student registration with live camera photo face capture & 128-D vector generation, teacher registration with institutional ID validation, and email OTP verification.
teacher_attendance/routes.py (Faculty Punch)	777 lines 32.3 KB	**13 Endpoints:** • recalculate_daily_status() • haversine_distance() • verify_teacher_face() • mark_morning_attendance() • mark_evening_attendance()	Faculty Daily Attendance & Geofencing Engine: tracks teacher daily punch-in/out with facial biometric verification, validates physical presence using Haversine GPS geofencing radius around campus, enforces morning late/grace rules, and auto-marks uninformed absences after cut-off times.

5. Database Schema & Entity Relationship Architecture

The database contains **31 relational models** structured with strict foreign key constraints, indexes, and cascading rules across 6 functional domains:

Domain	Key Models & Tables	Primary Keys & Foreign Key Relationships	Domain Purpose & Business Logic
1. Identity & Profiles	User Teacher Student Department IssuedTeacherID	• User.id (PK) • Teacher.user_id -> User.id • Student.user_id -> User.id • Student.class_id -> Class.id	Manages user credentials (PBKDF2 passwords), role scoping ('admin', 'teacher', 'student'), student roll numbers, and raw 128-D binary facial encodings (LargeBinary).
2. Master Academics	Class Subject TeacherAssignment	• Class.class_teacher_id -> Teacher.id • Subject.class_id -> Class.id • Subject.teacher_id -> Teacher.id	Maintains institutional academic hierarchy: classrooms, subject codes, assigned subject faculty, and designated class teachers.
3. Scheduling & Daily Engine	Timetable DailySchedule Holiday TimetablePeriodSetting	• DailySchedule.timetable_id -> Timetable.id • DailySchedule.substitute_teacher_id -> Teacher.id	Stores recurring weekly master timetable and date-specific daily schedule instances tracking proxy substitutes, leaves, and class cancellations.
4. Attendance Sessions	AttendanceSession AttendanceRecord AttendanceAuditLog	• AttendanceRecord.session_id -> AttendanceSession.id (CASCADE) • AttendanceRecord.student_id -> Student.id	Records live/completed class lecture attendance sessions, individual student statuses (PRESENT / ABSENT), confidence scores, and audit logs.
5. Faculty Biometrics	TeacherDailyAttendance TeacherLeave TeacherOfficeLocation	• TeacherDailyAttendance.teacher_id -> Teacher.id • TeacherLeave.teacher_id -> Teacher.id	Tracks daily faculty punch-in/out times, morning late minutes, uninformed absence flags, leave applications, and campus GPS geofence coordinates.
6. Governance & Notices	CorrectionRequest StudentEditRequest ClassAnnouncement StudentDismissedNotice	• CorrectionRequest.session_id -> AttendanceSession.id • ClassAnnouncement.class_id -> Class.id	Enforces formal approval chains for locked attendance corrections, student profile updates, and real-time class cancellation broadcasts.

6. Frontend Structure & 41 Jinja2 Template Breakdown

The user interface comprises **41 responsive HTML5/Bootstrap templates** organized into role-based views inheriting from a shared master layout (base.html):

Role / Category	Template File Names	Key UI Components & User Experience Flow
Admin Subsystem (17 Templates)	dashboard.html admin_emergency_proxy_desk.html admin_approvals.html admin_teacher_attendance.html manage_timetable.html manage_classes.html manage_subjects.html manage_holidays.html retention_risk_report.html attendance_audit_logs.html export_exam_eligibility.html	• Institutional pulse cards with live KPI metrics. • Emergency Proxy Desk with live unassigned slots and compact history logs. • Multi-tab institutional approvals (leaves, student profile edits, corrections). • Visual timetable grid manager with drag-and-drop slot creation. • Defaulter retention risk analytics and CSV/Excel export engines.
Teacher Subsystem (11 Templates)	teacher_dashboard.html teacher_take_attendance.html teacher_take_proxy_attendance.html teacher_proxy_classes.html teacher_enrolled_students.html teacher_leave_applications.html teacher_my_attendance.html teacher_reports.html class_teacher_dashboard.html	• Real-time WebRTC camera scan with dynamic face bounding box canvas. • Interactive manual roll checkbox review table with live search. • Session Attendance Log with one-click WhatsApp attendance report sharing. • Proxy duty classroom launcher with substitute attendance controls. • Leave request submission and daily biometric punch history.
Student Subsystem (6 Templates)	student_dashboard.html student_notices.html student_profile_edit.html student_request_modification.html student_register_photo.html	• Subject-wise attendance progress rings with 75% exam eligibility indicators. • Formal attendance discrepancy reporting with proof document uploads. • Dynamic broadcast feed for timetable updates and cancelled lectures. • Self-service profile modification request forms.
Authentication & Shared (7 Templates)	landing.html base.html admin_login.html reset_password.html partials/nav.html partials/footer.html	• Hero landing portal with biometric feature showcases. • Master responsive layout (base.html) with CSS variables, dark/light theme switcher, Toast notifications, and modal managers. • Single Sign-On and multi-role login forms with password reset workflows.

7. Environment Configuration, Deployment & Production Recipes

7.1 Environment Variables Reference (.env):

SECRET_KEY	Cryptographic session salt for cookie signing and CSRF tokens.
DATABASE_URL	sqlite:///smartvision.db (Local) or postgresql://user:pass@host/db (Cloud).
MAIL_SERVER / MAIL_PORT	smtp.gmail.com / 587 (TLS enabled for transactional emails).
MAIL_USERNAME / PASSWORD	Institutional sender Gmail and Google 16-character App Password.
GOOGLE_CLIENT_ID / SECRET	Google Cloud OAuth 2.0 credentials for institutional Single Sign-On.
PORT / FLASK_DEBUG	Port 7860 / Debug 0 (Production) or 1 (Hot-Reloading).

7.2 Multi-Cloud Deployment Guide:

- **Render / Railway (PaaS):** Connect GitHub repository. Set build command to pip install -r requirements.txt and start command to gunicorn -w 2 -b 0.0.0.0:7860 --timeout 120 app:app.
- **AWS EC2 / DigitalOcean (VPS):** Ubuntu 22.04 LTS instance with Nginx reverse proxy passing traffic to Gunicorn socket on port 7860.
- **WebRTC HTTPS Requirement:** WebRTC camera streams (getUserMedia) are strictly blocked by browsers on non-localhost HTTP. Production domains must install an SSL certificate via Certbot: sudo certbot --nginx -d yourdomain.com.

7.3 Production Dockerfile Build Recipe:

```
FROM python:3.11-slim RUN apt-get update && apt-get install -y --no-install-recommends \ build-essential cmake pkg-config libgl1-mesa-glx libglib2.0-0 \ libx11-dev libgtk-3-dev && rm -rf /var/lib/apt/lists/* WORKDIR /app COPY requirements.txt . RUN pip install --no-cache-dir -r requirements.txt COPY . . EXPOSE 7860 CMD ["gunicorn", "-w", "2", "-b", "0.0.0.0:7860", "--timeout", "120", "app:app"]
```

8. Mathematical Formulations & External Examiner Viva Cheat Sheet

8.1 Core Mathematical Formulations:

Metric Loss (Triplet Loss):	`L = sum max(0, ||f(anchor) - f(positive)||^2 - ||f(anchor) - f(negative)||^2 + 0.2)`
128-D Euclidean Distance:	`d(e_u, e_s) = sqrt( sum_{i=1}^{128} (e_{u,i} - e_{s,i})^2 ) --> Match if d < 0.55`
Strict Attendance %:	`Attendance % = ( COUNT(records == PRESENT) / COUNT(sessions == COMPLETED) ) * 100`
Haversine Geofencing:	`d = 2R * arcsin( sqrt( sin^2(dlat/2) + cos(lat1)*cos(lat2)*sin^2(dlon/2) ) ) <= Radius`

8.2 External Faculty Viva Voce High-Impact Q&A; Cheat Sheet:

Q1: Why use Flask over Django?

Flask is an ultra-lightweight WSGI microframework. Django's heavy ORM adds unnecessary latency during 350ms multi-face frame streams. Flask Blueprints gave us clean isolation while allowing direct C-bindings to NumPy, Dlib, and OpenCV.

Q2: How does the system handle multi-face recognition in a single classroom frame?

Dlib's HOG frontal detector locates all faces simultaneously. Each face chip is aligned via 68 landmarks and mapped to a 128-D vector by ResNet-34. NumPy calculates vectorized Euclidean distance matrices against all class student vectors in <1ms.

Q3: How are face vectors stored efficiently in the database?

Instead of storing raw images, we serialize the 128-float vector (np.float64) into a 1,024-byte binary BLOB (db.LargeBinary). This cuts storage by 99.8% and enables instant in-memory deserialization via np.frombuffer().

Q4: How does the Emergency Proxy Desk work?

The engine cross-references daily teacher attendance with the active timetable. If an absent teacher has a class in Period 3, it filters all present teachers, eliminates those who are busy, ranks free teachers by availability, and allows 1-click proxy allocation with live student broadcast notices.

Q5: How is anti-spoofing and proxy prevention enforced?

We enforce 4 security layers: (1) Live WebRTC camera pulse streams instead of static photos, (2) GPS Haversine radius geofencing for faculty check-in, (3) Same-day 11 PM session lockouts, and (4) Comprehensive admin/teacher audit logs.